

Cyber Attack Detection Rules

Python backend reference for the simulated website-traffic stream

Purpose: Implement deterministic detection signals and correlation for the seven attack scenarios generated by `web_traffic_sim.py`. The simulator also creates two benign anomalies that must not automatically trigger attack alerts.

Detection philosophy: Do not build one giant rule per attack. Extract small signals from events, maintain short rolling windows, then correlate signals into an attack type and Attention Score. This makes the detector more robust to legitimate unusual traffic.

#	Scenario	Primary signals
1	Brute Force	Repeated failed logins; same user/IP; success after failures
2	Credential Stuffing	Many accounts targeted; distributed login attempts
3	Account Takeover	New IP/UA; successful login; sensitive actions afterward
4	Web Scan / Probing	Sensitive paths; error spike; many unique paths; injection patterns
5	Data Exfiltration	Large bytes_out; API scraping; sequential pagination
6	Admin Abuse	Unusual admin login; suspicious commands; command → transfer
7	HTTP Flood	Request-rate spike; many IPs; latency/5xx degradation
B1	Flash Crowd	Legitimate 3–5× traffic spike — benign anomaly
B2	Nightly Backup	Expected backup process + large transfer — benign anomaly

1. Brute Force

The simulator creates fast and low-and-slow repeated failed login attempts against one victim account. It may eventually generate a successful login.

BF-1 — Failed logins per user

```
failed_logins(user, 5min) > 10
```

BF-2 — Failed logins per IP

```
failed_logins(ip, 5min) > 20
```

BF-3 — High failure rate

```
failure_rate(user, 5min) > 0.80
```

BF-4 — Success after failures

```
failed_logins(user, 10min) >= 8 AND successful_login(user) occurs after the failures
```

2. Credential Stuffing

The simulator attacks many different accounts, using several IPs, with relatively few attempts per account. Detection should therefore focus on account breadth rather than only request volume.

CS-1 — Many accounts from one IP

```
unique_users(ip, 5min) > 20
```

CS-2 — Broad failed-login campaign

```
failed_logins(ip, 5min) > 30 AND unique_users(ip, 5min) > 10
```

CS-3 — Distributed campaign

Multiple IPs target an overlapping population of users within a short window

3. Account Takeover

The simulator first creates legitimate activity for a user and later logs the same user in from a different network, followed by sensitive account activity.

ATO-1 — New IP

```
successful_login(user) from an IP not recently associated with that user
```

ATO-2 — New IP + sensitive action

```
new_ip(user) AND successful_login(user) AND sensitive_action within 10min
```

ATO-3 — New IP + new UA + high activity

```
new_ip(user) AND new_user_agent(user) AND successful_login(user) AND high_activity(user)
```

4. Web Scan / Probing

The simulator generates rapid requests for sensitive paths plus SQL injection, XSS, and path-traversal-style probes. It also produces mostly 404/403/500 responses.

WS-1 — Sensitive path

```
path matches known sensitive endpoints such as /.env, /.git/config, /phpmyadmin/, /actuator/env
```

WS-2 — Error spike

```
errors(ip, 1min) > 20 AND error_rate(ip, 1min) > 0.70
```

WS-3 — Many unique paths

`unique_paths(ip, 1min) > 30`

WS-4 — Injection signature

`path/query` matches `SQLi`, `XSS`, or traversal patterns

Suggested signature lists

```
SENSITIVE_PATHS = [  
    "/.env", "/.git/config", "/wp-admin/", "/phpmyadmin/",  
    "/server-status", "/actuator/env", "/backup.zip"  
]
```

```
SQLI_PATTERNS = [  
    "' OR '1'='1", "UNION SELECT", "DROP TABLE", "SLEEP("  
]
```

```
XSS_PATTERNS = ["<script", "javascript:"]  
TRAVERSAL_PATTERNS = [ "../", "%2f..%2f"]
```

5. Data Exfiltration

The simulator creates large admin exports or scripted API scraping with many sequential pages.

EX-1 — Large transfer

`event_type == 'data_transfer' AND bytes_out > 100_000_000`

EX-2 — Abnormal aggregate transfer

`bytes_out(user, 10min)` exceeds a user-specific baseline

EX-3 — API scraping

`api_requests(user, 5min) > 50`

EX-4 — Sequential pagination

`sequential_pages(user) > 20`

EX-5 — Suspicious login → transfer

`successful_login(unusual_context)` followed by `large_data_transfer`

6. Admin Abuse

The simulator schedules this scenario at night, uses an admin account, runs suspicious commands, and then creates a large transfer.

AA-1 — Unusual admin login

`admin login` outside expected working hours OR from an unusual admin IP

AA-2 — Suspicious process

`process` matches commands such as `'cat /etc/shadow'`, `'curl ... | sh'`, `'useradd'`, `'crontab'`, `'nc -e'`

AA-3 — Privileged activity → transfer

`admin_login AND suspicious_process AND large_data_transfer` within a correlation window

7. HTTP Flood

The simulator creates 800–2500 requests from roughly 30–120 IPs over 30–90 seconds and progressively increases server degradation.

HF-1 — Request-rate spike

`requests_per_second > baseline * 5`

HF-2 — Distributed sources

`unique_ips_per_minute > normal baseline`

HF-3 — Server degradation

`response_ms increasing AND 5xx_rate increasing`

HF-4 — Combined flood

`request_rate spike AND unique_ips spike AND 5xx_rate above threshold`

Benign Anomalies — Must Not Automatically Alert

B1. Flash Crowd

The simulator deliberately creates a sudden 3–5× legitimate traffic increase. The presence of a traffic spike alone should therefore not equal DDoS.

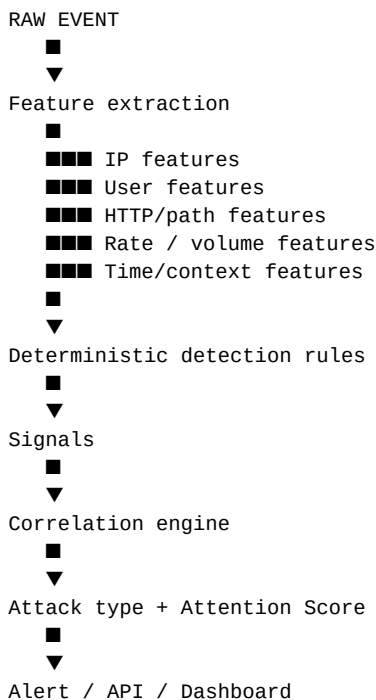
```
traffic_spike
AND normal_user_distribution
AND normal_paths
AND normal_success_rate
  → benign_anomaly / low attention
```

B2. Nightly Backup

The simulator creates a nightly pg_dump followed by a multi-gigabyte transfer. A large transfer alone should therefore not equal exfiltration.

```
user == "svc_backup"
AND process == "pg_dump --format=custom appdb"
AND known DB host
AND expected backup time
  → benign_anomaly / suppress exfiltration alert
```

Recommended Python Architecture



Attention Score Example

Use individual rules as signals rather than treating every signal as an immediate alert. For example, a new IP alone is weak; a new IP followed by sensitive activity and unusually high API activity is stronger.

Signal	Example points
New IP	+20
New User-Agent	+15
Unusual login context	+20
Sensitive action	+25

High API activity	+30
-------------------	-----

Implementation note: The thresholds above are starting points for the simulator, not production security thresholds. They should be configurable and tuned against generated traffic, false positives, and observed baselines.